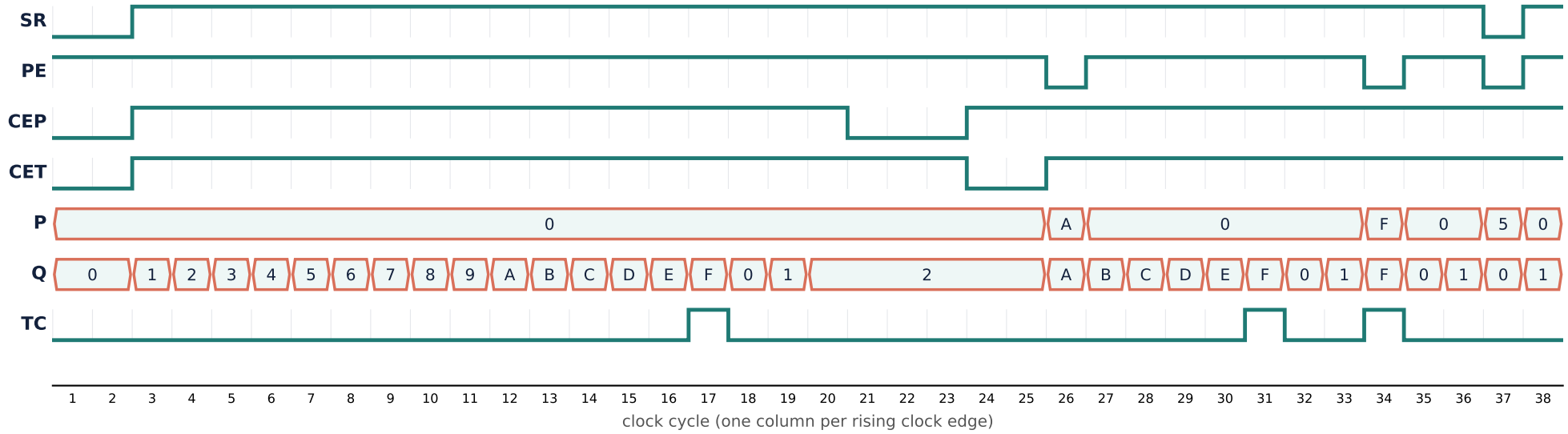
P1 74LS163 counter

AAC2M2P1.vhd - 4-bit synchronous counter, SR/PE active low

What you are seeing: cycles 1-2 reset Q to 0. Cycles 3-19 count 1,2,...,F; TC goes high only when Q = F and CET = 1, then Q wraps to 0. CEP=0 or CET=0 makes the counter hold. PE=0 loads P (A, then F). SR=0 beats PE=0 (last load attempt is ignored).



cycle	SR	PE	CEP	CET	P	Q	TC	what happens
1	0	1	0	0	0	0	0	reset
2	0	1	0	0	0	0	0	reset
3	1	1	1	1	0	1	0	count up
4	1	1	1	1	0	2	0	count up
5	1	1	1	1	0	3	0	count up
6	1	1	1	1	0	4	0	count up
7	1	1	1	1	0	5	0	count up
8	1	1	1	1	0	6	0	count up
9	1	1	1	1	0	7	0	count up
10	1	1	1	1	0	8	0	count up
11	1	1	1	1	0	9	0	count up
12	1	1	1	1	0	A	0	count up
13	1	1	1	1	0	B	0	count up

cycle	SR	PE	CEP	CET	P	Q	TC	what happens
14	1	1	1	1	0	C	0	count up
15	1	1	1	1	0	D	0	count up
16	1	1	1	1	0	E	0	count up
17	1	1	1	1	0	F	1	count up
18	1	1	1	1	0	0	0	count up
19	1	1	1	1	0	1	0	count up
20	1	1	1	1	0	2	0	count up
21	1	1	0	1	0	2	0	hold (CEP=0)
22	1	1	0	1	0	2	0	hold (CEP=0)
23	1	1	0	1	0	2	0	hold (CEP=0)
24	1	1	1	0	0	2	0	hold (CET=0)
25	1	1	1	0	0	2	0	hold (CET=0)
26	1	0	1	1	A	A	0	parallel load 1010

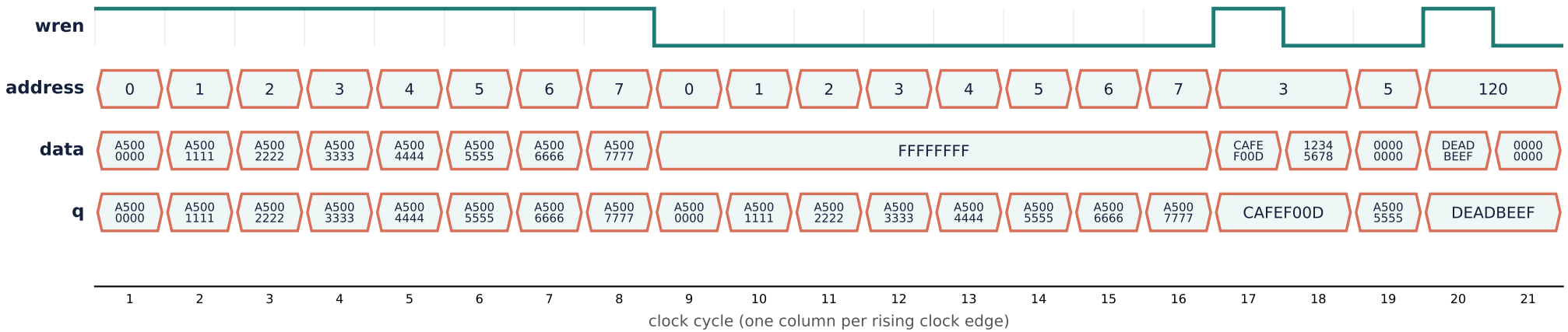
cycle	SR	PE	CEP	CET	P	Q	TC	what happens
27	1	1	1	1	0	B	0	count up
28	1	1	1	1	0	C	0	count up
29	1	1	1	1	0	D	0	count up
30	1	1	1	1	0	E	0	count up
31	1	1	1	1	0	F	1	count up
32	1	1	1	1	0	0	0	count up
33	1	1	1	1	0	1	0	count up
34	1	0	1	1	F	F	1	load 1111 (TC=1)
35	1	1	1	1	0	0	0	count: wraps to 0
36	1	1	1	1	0	1	0	count: wraps to 0
37	0	0	1	1	5	0	0	SR=0 wins over load
38	1	1	1	1	0	1	0	count up

Real GHDL simulation of the designs - for learning only, NOT the course myvectorh.out

P2 128 x 32 RAM

AAC2M2P2.vhd - clocked write, unregistered (asynchronous) read

What you are seeing: cycles 1-8 write A5000000, A5001111, ... into addresses 0-7. Cycles 9-16 read them back with wren=0 (the data bus is junk FFFFFFFF and is ignored). Cycle 17 overwrites address 3 with CAFEF00D; cycle 18 shows that changing data while wren=0 does not change memory. Address 120 is written and read at the end.



cycle	address	data	wren	q (read data)	what happens
1	0	A5000000	1	A5000000	write
2	1	A5001111	1	A5001111	write
3	2	A5002222	1	A5002222	write
4	3	A5003333	1	A5003333	write
5	4	A5004444	1	A5004444	write
6	5	A5005555	1	A5005555	write
7	6	A5006666	1	A5006666	write
8	7	A5007777	1	A5007777	write
9	0	FFFFFFFF	0	A5000000	read (data ignored; wren=0)
10	1	FFFFFFFF	0	A5001111	read (data ignored; wren=0)
11	2	FFFFFFFF	0	A5002222	read (data ignored; wren=0)

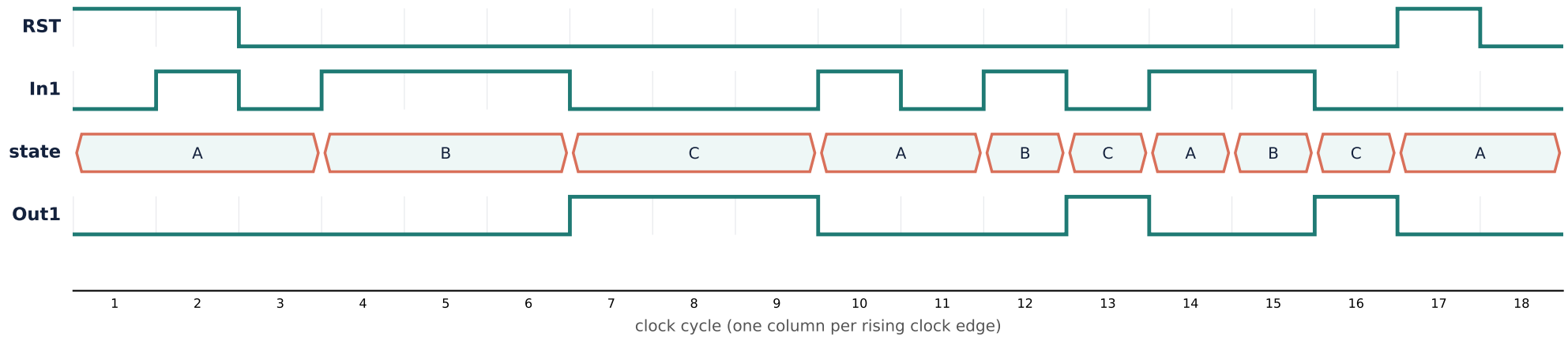
cycle	address	data	wren	q (read data)	what happens
12	3	FFFFFFFF	0	A5003333	read (data ignored; wren=0)
13	4	FFFFFFFF	0	A5004444	read (data ignored; wren=0)
14	5	FFFFFFFF	0	A5005555	read (data ignored; wren=0)
15	6	FFFFFFFF	0	A5006666	read (data ignored; wren=0)
16	7	FFFFFFFF	0	A5007777	read (data ignored; wren=0)
17	3	CAFEF00D	1	CAFEF00D	overwrite address 3
18	3	12345678	0	CAFEF00D	read 3 (wren=0; memory unchanged)
19	5	00000000	0	A5005555	read 5
20	120	DEADBEEF	1	DEADBEEF	write high address 120
21	120	00000000	0	DEADBEEF	read 120

Real GHDL simulation of the designs - for learning only, NOT the course myvectorh.out

P3 Finite State Machine

AAC2M2P3.vhd - Moore machine, states A/B/C, RST active high

What you are seeing: RST=1 forces state A. A goes to B when In1=1; B goes to C when In1=0; C goes back to A when In1=1; otherwise the state holds. Out1 is 1 only while the machine is in state C (the state column is tracked by the testbench for display).



cycle	RST	In1	state after edge	Out1	what happens
1	1	0	A	0	reset -> A
2	1	1	A	0	reset -> A
3	0	0	A	0	A stays A (In1=0)
4	0	1	B	0	A -> B
5	0	1	B	0	B stays B (In1=1)
6	0	1	B	0	B stays B
7	0	0	C	1	B -> C
8	0	0	C	1	C stays C (Out1=1)
9	0	0	C	1	C stays C
10	0	1	A	0	C -> A
11	0	0	A	0	A stays A
12	0	1	B	0	A -> B
13	0	0	C	1	B -> C
14	0	1	A	0	C -> A
15	0	1	B	0	A -> B
16	0	0	C	1	B -> C
17	1	0	A	0	RST while in C -> A
18	0	0	A	0	A stays A

Real GHDL simulation of the designs - for learning only, NOT the course myvectorh.out

H1 32-bit ALU

AAC2M2H1.vhd - combinational ALU, 8 opcodes

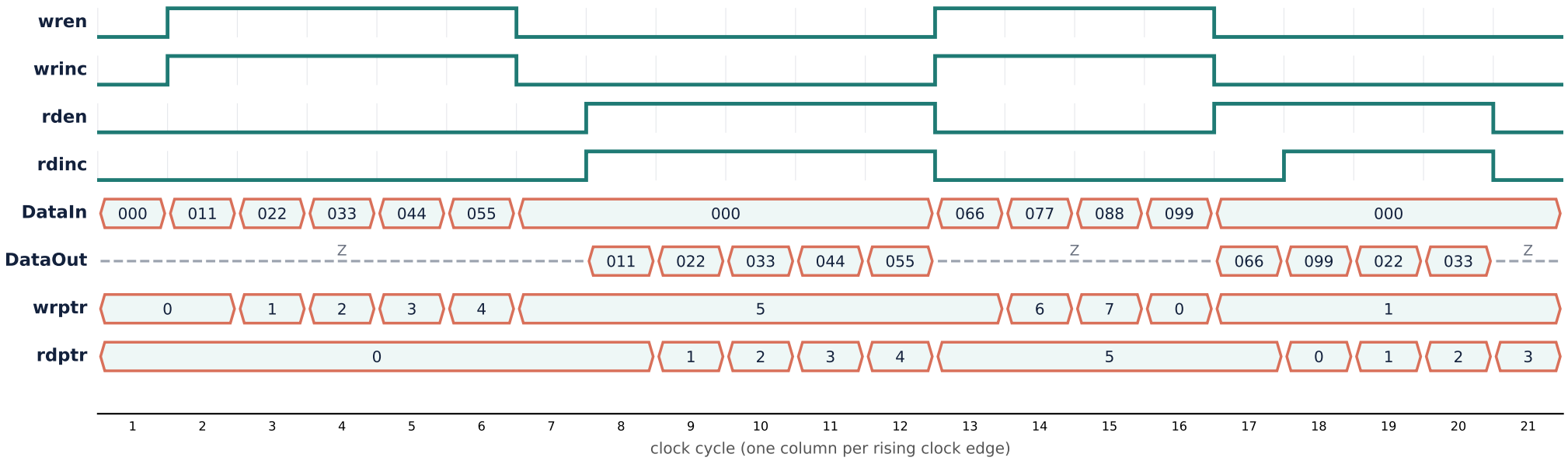
What you are seeing: the first 8 rows apply every opcode to A = 0x64 (100) and B = 0x19 (25): e.g. add gives 0x7D (125), subtract gives 0x4B (75), AND gives 0x00, OR gives 0x7D. The last rows show wrap-around behaviour (FFFFFFFF + 1 = 0, 0 - 1 = FFFFFFFF) and bit-wise AND / OR on bit patterns. Values are hexadecimal.

test	opcode	operation	A (hex)	B (hex)	Y (hex)	note
1	000	Y = A	00000064	00000019	00000064	A=100 and B=25
2	001	Y = A + B	00000064	00000019	0000007D	A=100 and B=25
3	010	Y = A - B	00000064	00000019	0000004B	A=100 and B=25
4	011	Y = A AND B	00000064	00000019	00000000	A=100 and B=25
5	100	Y = A OR B	00000064	00000019	0000007D	A=100 and B=25
6	101	Y = A + 1	00000064	00000019	00000065	A=100 and B=25
7	110	Y = A - 1	00000064	00000019	00000063	A=100 and B=25
8	111	Y = B	00000064	00000019	00000019	A=100 and B=25
9	001	Y = A + B	FFFFFFFF	00000001	00000000	add wraps to 0
10	010	Y = A - B	00000000	00000001	FFFFFFFF	subtract wraps to FFFFFFFF
11	101	Y = A + 1	FFFFFFFF	00000000	00000000	increment wraps to 0
12	110	Y = A - 1	00000000	00000000	FFFFFFFF	decrement wraps to FFFFFFFF
13	011	Y = A AND B	F0F0F0F0	0FF0FF00	00F0F000	AND of bit patterns
14	100	Y = A OR B	F0F0F0F0	0FF0FF00	FFF0FFF0	OR of bit patterns

H2 FIFO 8 x 9

AAC2M2H2.vhd - 8 registers of 9 bits, tri-state output

What you are seeing: pointers are cleared, then 011,022,033,044,055 (hex) are written (wrptr 0->5). With rden=0, DataOut is high-impedance (Z). Reading with rden=1 and rdinc=1 returns the words in the same order they were written. Four more writes fill the registers and the last one wraps around to register 0, overwriting the oldest word; RdPtrClr then starts reading again from register 0 (the new value 099).



cycle	wren	wrinc	rden	rdinc	DataIn	DataOut	wrptr	rdptr	what happens
1	0	0	0	0	000	ZZZ	0	0	clear both pointers
2	1	1	0	0	011	ZZZ	0	0	write 17 and wrinc
3	1	1	0	0	022	ZZZ	1	0	write 34 and wrinc
4	1	1	0	0	033	ZZZ	2	0	write 51 and wrinc
5	1	1	0	0	044	ZZZ	3	0	write 68 and wrinc
6	1	1	0	0	055	ZZZ	4	0	write 85 and wrinc
7	0	0	0	0	000	ZZZ	5	0	rden=0 -> DataOut is Z
8	0	0	1	1	000	011	5	0	read + rdinc
9	0	0	1	1	000	022	5	1	read + rdinc
10	0	0	1	1	000	033	5	2	read + rdinc
11	0	0	1	1	000	044	5	3	read + rdinc

cycle	wren	wrinc	rden	rdinc	DataIn	DataOut	wrptr	rdptr	what happens
12	0	0	1	1	000	055	5	4	read + rdinc
13	1	1	0	0	066	ZZZ	5	5	write 102 and wrinc
14	1	1	0	0	077	ZZZ	6	5	write 119 and wrinc
15	1	1	0	0	088	ZZZ	7	5	write 136 and wrinc
16	1	1	0	0	099	ZZZ	0	5	write 153 and wrinc
17	0	0	1	0	000	066	1	5	RdPtrClr (rdptr goes to 0 at the edge)
18	0	0	1	1	000	099	1	0	read + rdinc
19	0	0	1	1	000	022	1	1	read + rdinc
20	0	0	1	1	000	033	1	2	read + rdinc
21	0	0	0	0	000	ZZZ	1	3	rden=0 -> Z